



Jan Vitek

Examples

```
With types: df: { revenue : dbl, ... } has no field `reveneu`, rejects df$reveneu call
```

- R hides stuff: a value's storage type (int vs dbl), its shape (scalar vs vector vs matrix), its S3 classes, these are all invisible at the call site.
- **Types make the invisible bits explicit where needed!**

- Infer types automatically so user only annotate when needed

```
Vector types (vec):  raw | lgl | int | dbl | cplx | chr
Scalar types (vec1): raw1 | lgl1 | int1 | dbl1 | cplx1 | chr1
Singleton types:    42 | "hello world" | tt | ff | ...
```

```
List of (any number of) double vectors: { dbl }
Exactly two named elements: { x : dbl, y : dbl }
Two named elements and anything else: { x : dbl, y : dbl, any }
```

- can decorate any type `{ id : int, name : chr } <data.frame ...>`
- any boolean combination `<matrix, ~array ...>`
- for simplicity, types do not capture the order of classes

Intersection of arrows: $((x: \text{dbl}) \rightarrow (\text{dbl})) \ \& \ ((x: \text{int}) \rightarrow (\text{dbl})) \ \& \ ((x: \text{chr}) \rightarrow (\text{chr}))$

- The mathematical foundations we rely on
- Support set connectives and polymorphism
- Based on recent formal works: [1, 2, 3, 4]
- Library: <https://e-sh4rk.github.io/MLsem/>

Type error argument `x` expects `dbl`, but `species` is `int<factor>` — a factor is disjoint from `dbl`.

```
typr(shape-mismatch) · rescale.R [8, 9]
```

R expression	Type	Comment
NULL	null	its own type, distinct from empty vectors
list(1, 2, 3)	{ ; dbl }	homogeneous list of doubles
list(1, "a", TRUE)	{ ; dbl chr lg1 }	heterogeneous — union element type
list(x = 1, y = 2)	{ x : dbl, y : dbl }	record with named fields
list(x = 1, y = "a", 3)	{ x : dbl, y : chr; any }	named fields plus anything else
factor(c("a", "b", "a"))	int<factor>	codes are int; class makes it disjoint from dbl
ordered(c("lo", "hi"))	int<ordered, factor ...>	carries two classes at once
matrix(1:6, 2, 3)	int<matrix, array ...>	shape is the type bit R hides
data.frame(id = 1:3, name = c("a", "b", "c"))	{ id : int, name : chr }<data.frame ...>	list shape plus S3 class together
lm(y ~ x)	{ coefficients : dbl, residuals : dbl ; any}<lm>	a big record with a class
function(x) x + 1	(x: dbl)->dbl & (x: clx)->clx & (x: clx1)->clx1	increment function
nrow	(x: vec <array ...> <data.frame ...>) -> int	nrow function – can take vectors, arrays, df
df\$col	(x: { #col: 'a ...'}<data.frame ...>, #col) -> 'a	extraction of a dataframe column

Type of c(.,.)	raw	logical	integer	double	complex	character
raw	raw	logical	integer	double	complex	character
logical	logical	logical	integer	double	complex	character
integer	integer	integer	integer	double	complex	character
double	double	double	double	double	complex	character
complex	complex	complex	complex	complex	complex	character
character	character	character	character	character	character	character

```
raw1 <  lg1 <  int1 <  db1 <  clx <  chr1
  ^      ^      ^      ^      ^      ^
raw <  lg <  int <  db <  clx <  chr
```

```
df : { id:dbl ...}<data.frame ...>
df <- data.frame(id = 1L:3L) # OK, id is int < dbl
df <- data.frame(id = 1:3, name = c("a", "b", "c")) # OK, type of df allows extra columns
df <- data.frame(id = c("a", "b", "c")) # NOT OK, id is chr > dbl
```

```
typeof : ( x : { any } ) -> "list"      rbind : ( ... : <data.frame ...> ) -> <data.frame ...>
typeof : ( x : null ) -> "NULL"         rbind : ( ... : <matrix ...> ) -> <matrix ...>
typeof : ( x : lgl ) -> "logical"       rbind : ( ... : vec<> ) -> <matrix>
```

```
Identity : (x : 'a) -> 'a          (+) : (x : int, y : int) -> int
[.] : ({'a}, dbl) -> 'a          (+) : (x : dbl, y : dbl) -> dbl
[.] : (v('a), dbl) -> v1('a)    (+) : (x : chr, y : chr) -> chr
```

Type variables (`'a', 'b'`): capture a single type. Row variables (``r``): capture a list of types.

```
lapply : (X : v('a), FUN : @(v1('a), ... : `r) -> 'b, ... : `r) -> {'b}
lapply : (X : {'a}, FUN : @( 'a , ... : `r) -> 'b, ... : `r) -> {'b}
```

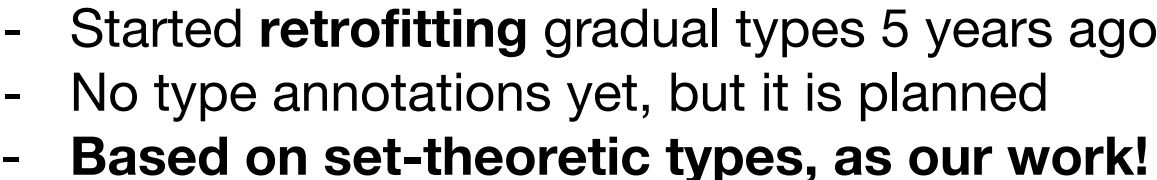
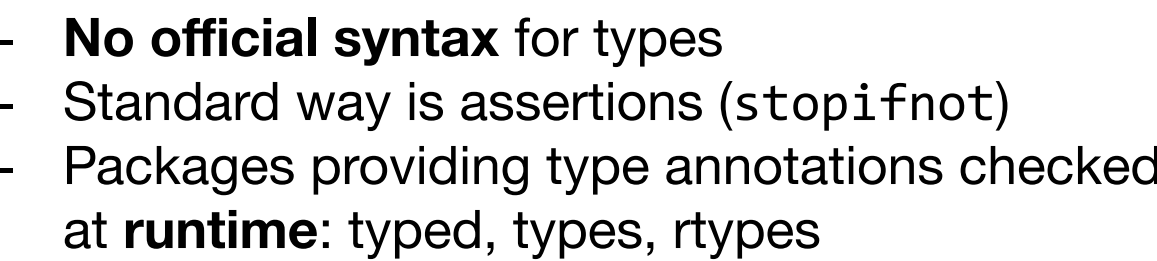
```

      Exploit function signature
neg <- function(x: null | lg1 | dbl) -> null | lg1 | dbl {
  if (typeof(x) == "NULL") return(NULL)
  if (typeof(x) == "logical") return(!x)
  return(-x)
}

```

The rest is typed under the assumption that
 $x : \sim \text{NULL}$

- Statically-typed variant of JS, **widely adopted**
- A **new syntax** for types on top of JS
- Type annotations are **erased**, yielding valid JS



<https://github.com/PRL-PRG/typr>